1. Explain what a cross-cutting concern is, and use the OrderService example from class to explain"tangling" and "scattering".

A cross-cutting concern is a concern, such as logging or security, that affects multiple parts of an application but is not part of the core business logic.

Using the OrderService example, tangling occurs when business logic is mixed with cross-cutting concerns, such as logging and security checks, in the same method. Scattering occurs when the same cross-cutting concern, such as logging, is duplicated across multiple classes, such as OrderService, PaymentService, and UserService.

Spring AOP helps separate these cross-cutting concerns from the main business logic.

2. Define aspect, join point, pointcut, advice, target and proxy, and state what a join point always is inSpring AOP.

Aspect: A module that encapsulates a cross-cutting concern, such as logging or security.

Join point: A point during program execution where advice can be applied. In Spring AOP, a join point always represents a method execution.

Pointcut: An expression that selects which join points should receive advice.

Advice: The action performed at a selected join point.

Target: The original object containing the business logic that is being advised.

Proxy: An object that intercepts method calls and applies advice before, after, or around calls to the target object.

3. Compare the five advice types (@Before, @AfterReturning, @AfterThrowing, @After, @Around):when each one runs, and which of them can change the return value or prevent the target method from running.

@Before: Runs before the target method executes. It cannot directly change the method's return value or normally skip the target method, although throwing an exception can prevent execution.

@AfterReturning: Runs after the target method completes successfully. It cannot directly replace the returned value.

@AfterThrowing: Runs when the target method exits by throwing an exception. It is used for exception-related handling and does not directly change the return value.

@After: Runs after the target method finishes, whether it returns normally or throws an exception. It is similar to a finally block.

@Around: Runs around the target method invocation. It can change the returned value by returning a different result, and it can prevent the target method from running by not calling proceed().

Only @Around can directly control whether the target method is invoked and replace its return value through its return statement.

4. Describe three mistakes that are easy to make inside an @Around advice and the effect of each one.

a.Forgetting to call proceed(): The target method never executes, so the intended business operation is skipped.

b.Calling proceed() more than once: The target method may execute multiple times, potentially causing duplicate operations such as creating multiple orders.

c.Returning the wrong value: If the advice returns null or another incorrect value instead of the target method's result, the caller receives an incorrect result.

To avoid these mistakes, call proceed() the intended number of times, store its result, and return that result unless you deliberately want to replace it.

5. Explain what each pointcut expression matches:

execution(* com.example.orders.service.*.*(..)): Matches any method in classes directly inside the service package, with any return type and parameters.
execution(* com.example.orders..*.*(..)): Matches any method in the orders package or its subpackages.
execution(* *(Long, ..)) : Matches methods whose first parameter is Long, followed by zero or more parameters.
and@annotation(com.example.orders.aop.LogExecutionTime): Matches methods annotated with LogExecutionTime.

6. Explain how Spring AOP applies advice through proxies, at which step of the bean lifecycle the proxy is created, and how you can check that an injected bean is a proxy.
Spring AOP uses proxies to intercept method calls and apply advice. Proxies are typically created by an auto-proxy creator, a BeanPostProcessor, during post-initialization processing. Use AopUtils.isAopProxy(bean) to check whether an injected bean is a proxy.

7. Explain why calling a method annotated with @LogExecutionTime or @Transactional from another method of the same class skips the advice, and describe two ways to fix it.
Self-invocation bypasses advice because an internal method call does not go through the Spring proxy. Two fixes are:
1. Move the advised method to another Spring bean and call it through the injected bean.
2. Call the method through the current bean's Spring proxy instead of directly.

8. State the default rollback rules of @Transactional, and describe three situations in which a @Transactional method does not roll back even though something went wrong.
By default, @Transactional rolls back on RuntimeException and Error, but not on checked exceptions.Three situations where rollback may not happen are:
1.A checked exception is thrown.
2.The exception is caught and not rethrown.
3.The failure occurs in a self-invocation that bypasses the Spring proxy, so the transaction advice is not applied.

9. SecurityAspect has @Order(1) and TimingAspect has @Order(2), and both use @Around on the same method. Describe the order in which their code runs. Then compare Spring AOP with AspectJ in terms of weaving and the join points each one supports.
SecurityAspect has @Order(1) and TimingAspect has @Order(2). The security aspect has higher precedence and runs first on entry. The timing aspect runs next. After the target method finishes, the advice executes in reverse order: TimingAspect exits first, then SecurityAspect.
Spring AOP uses runtime proxies to apply advice and mainly supports method-execution join points. AspectJ supports compile-time, post-compile, or load-time weaving and provides more join points, including method calls, field access, and constructor execution.